



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
INGENIERÍA EN INFORMÁTICA
ASIGNATURA: INGENIERÍA DE SOFTWARE II

INFORME TÉCNICO: GESTIÓN DE RIESGOS Y DEUDA
TÉCNICA EN SISTEMAS MODERNOS

Profesor:

Marquez Felix

Estudiantes:

Alondra Moreno CI: 30.331.870

Samuel Cortez CI: 27.158.842

Carlos Hernández CI: 27.975.753

Eduardo Quintero CI: 32.004.965

Ciudad Guayana, Septiembre 2026

Introducción

La gestión de riesgos en la ingeniería de software ha dejado de ser una actividad burocrática y estática para convertirse en una disciplina dinámica e indispensable. Históricamente, los riesgos se analizaban al inicio del proyecto en extensas matrices que luego eran olvidadas, lo cual resultaba ineficiente en entornos de alta incertidumbre. Hoy en día, evolucionar hacia una gestión proactiva e integrada dentro de las metodologías ágiles es crucial para la supervivencia del software.

Este informe aborda la aplicación de técnicas modernas como el Risk Storming y la gobernanza ágil, demostrando cómo los riesgos (desde la deuda técnica hasta el sesgo de inteligencia artificial) no solo deben ser identificados colaborativamente, sino que deben transformarse en tareas procesables y priorizables dentro del flujo de trabajo continuo (Backlog) de los equipos de desarrollo.

1. Desarrollo del Ejercicio 1: Risk Storming y Riesgos Modernos en una Fintech

Contexto: Aplicación móvil de préstamos alojada en arquitectura Serverless, utilizando una API de Inteligencia Artificial de un tercero para scoring crediticio.

Identificación de 4 Riesgos Específicos:

- **Técnico / Deuda Técnica:** Caída de la API del tercero o alta latencia extrema (Cold Starts en el entorno serverless) en momentos de pico de transacciones, dejando la app inoperativa sin un mecanismo de respaldo (Fallback) para aprobar créditos.
- **Ciberseguridad / Cadena de Suministro:** Dependencia de bibliotecas o SDKs cerrados del proveedor de IA que podrían contener vulnerabilidades no detectadas (zero-days), permitiendo una inyección de datos o filtración de información financiera sensible (ataque a la cadena de suministro).
- **Vendor Lock-in:** Acoplamiento profundo del código a los servicios propietarios del proveedor Cloud actual (por ejemplo, funciones Lambda específicas de AWS). Si el proveedor sube agresivamente los precios, la migración a otra nube sería extremadamente costosa y lenta.
- **Ético o Legal (IA):** El modelo de IA del tercero podría estar entrenado con datos sesgados, rechazando préstamos sistemáticamente a grupos demográficos vulnerables, lo que desataría demandas legales por discriminación y daño reputacional masivo al no poder explicar el 'código negro' de la IA.

Análisis Cualitativo del Riesgo Crítico:

Riesgo Seleccionado: Sesgo algorítmico y falta de explicabilidad legal en la aprobación de préstamos (Riesgo Ético/Legal).

- Probabilidad: 3 (Posible - Los modelos comerciales de IA de terceros a menudo sufren de sesgos no documentados).
- Impacto: 5 (Catastrófico - Demandas legales por discriminación crediticia, multas de entes reguladores y pérdida total de confianza del mercado).
- Risk Score (P x I): 15 (Riesgo Alto).

Justificación: Aunque la caída de un servidor (riesgo técnico) molesta temporalmente al usuario, una demanda por discriminación financiera basada en IA tiene el impacto suficiente para llevar a la Fintech a la bancarrota por violaciones regulatorias.

Estrategia de Respuesta (Mitigación):

Estrategia: MITIGAR. Se creará una validación estadística en paralelo y auditorías trimestrales a las decisiones de la IA.

Ejecución Ágil: Transformación a Spike en el Sprint Backlog:

- SPIKE: 'Investigar e implementar un middleware de validación estadística que intercepte y analice las decisiones de la API de IA para detectar sesgos demográficos antes de emitir un veredicto de crédito al usuario final'.
-
-
-
-
-
-
-

- **2. Desarrollo del Ejercicio 2: Registro de Riesgos Híbrido y Gobernanza Ágil**

Registro de Riesgos Híbrido

ID	Descripción del Riesgo	Prob	Impact	Dueño (Risk Owner)	Estado
R-01	Latencia o caída de API de terceros para créditos.	4	5	Líder Backend	Mitigando
R-02	Vendor Lock-in por Serverless propietario.	3	4	Arquitecto Cloud	Monitoreando
R-03	Vulnerabilidades en dependencias open-source.	3	3	Ingeniero QA	Mitigado
R-04	Sesgos algorítmicos en IA de préstamos.	2	5	Product Owner	Aceptado

Gobernanza Ágil: Mantenimiento del Registro sin Burocracia

En agilidad, un registro de riesgos no es un PDF olvidado, sino un documento vivo (o tablero virtual) que se audita constantemente. Para mantenerlo sin burocracia, se integra en los eventos de Scrum:

- **Integración en Sprint Planning:** El equipo revisa brevemente los riesgos con alta probabilidad de afectar el próximo entregable. Si un riesgo es inminente, se crean Historias de Usuario para mitigarlo dentro de ese mismo sprint.
- **Integración en Sprint Retrospective:** El equipo reflexiona sobre nuevos riesgos descubiertos durante el desarrollo de la semana (ej. deuda técnica que frenó el avance) y se actualiza el registro con nuevas lecciones aprendidas.

Ejemplo Práctico de Transformación:

Riesgo Original: Incremento imprevisto en los costos o cambio de condiciones de la API de IA.

Transformación (Enabler Story):

Como Arquitecto de Software, quiero implementar un sistema de almacenamiento en caché para las solicitudes duplicadas a la API de scoring, para reducir el volumen de llamadas de pago en un 40% y mitigar el aumento de tarifas del proveedor.

3. Desarrollo del Ejercicio 3: Comparativa de Métodos de Identificación

Cuadro Comparativo

Dimensión	Identificación Tradicional (Project Manager solo)	Risk Storming (Equipo Multifuncional)
Visión del Riesgo	Limitada a la perspectiva gerencial (presupuesto/tiempo).	Holística (técnica, de negocio, diseño y seguridad).
Dinámica	Estática, basada en listas de chequeo predefinidas de proyectos pasados.	Colaborativa y visual, basada en la arquitectura real (diagramas de componentes).
Descubrimiento de Deuda Técnica	Prácticamente nulo, el PM desconoce los problemas internos del código.	Alto, los programadores revelan los riesgos arquitectónicos ocultos.

Justificación Técnica de Superioridad del Risk Storming

El Risk Storming es sustancialmente superior porque el desarrollo de software es una labor multidisciplinaria compleja. Un Project Manager aislado, usando plantillas genéricas, jamás detectará un cuello de botella en una base de datos ni una inyección SQL. Al reunir a arquitectos, programadores, especialistas QA y DevOps frente a un diagrama arquitectónico (como en el Risk Storming), el equipo revela orgánicamente los 'agujeros negros' del sistema, detectando fallas lógicas, dependencias frágiles y deuda técnica que solo conocen los expertos que tocan el código a diario.

Conclusiones

Los riesgos no deben tratarse como un documento burocrático estático, sino que deben estar vivos e integrados directamente en el Product Backlog (mediante Spikes o Enablers) para ser procesados ágilmente.

En proyectos de tecnología moderna (como Cloud e IA), los riesgos más catastróficos ya no son solo retrasos de cronograma, sino elementos complejos como la vulnerabilidad de la cadena de suministro o los sesgos éticos algorítmicos.

La inteligencia colectiva de un equipo multifuncional (Risk Storming) es la única herramienta técnica válida y capaz de descubrir amenazas estructurales y deuda técnica que los enfoques gerenciales tradicionales ignoran por completo.